

4. Risks & Mitigations

4.1 Execution Dispatcher poison-pill. Risk: transient crashes recover safely via full re-scan (1.5), but if something about *today's specific data* deterministically crashes the Dispatcher, that loop repeats forever and no plans execute that day — silently. Mitigation: alert if today's tick isn't fully dispatched by a defined cutoff (e.g. 06:00 UTC) and page on-call.

4.2 Trading/Orders extended outage. Risk: `order-commands` durably queues and drains on recovery — no data loss — but there's no timeout or user-facing signal if an outage lasts hours/days; Executions sit `Initiated` indefinitely with no indication anything is wrong. Mitigation: alert on order-command age/consumer lag; consider a “still processing” status surfaced to the user past a threshold.

4.3 Non-InitiateExecution consumers aren't explicitly idempotent. Risk: Kafka delivery is at-least-once everywhere, not just there (2.5) — a redelivered `OrderFilled/OrderRejected` could double-apply if `UpdateOrderStatus` doesn't check current status first. Mitigation: every command handler must transition only from the expected prior status and treat a repeat as a no-op — a general invariant, not a one-off.

4.4 No override for a stuck idempotency key. Risk: the `(plan_id, execution_month)` uniqueness constraint (2.5) also blocks a legitimate manual re-run (e.g. support fixing a missed month). Mitigation: needs an admin/support runbook to reset or bypass the key for a specific plan+month, with audit logging — currently undesigned.

4.5 Outbox backlog growth. Risk: if Debezium lags or goes down, outbox rows accumulate unpublished, silently stalling projections and Notification. Mitigation: monitor backlog size and oldest-unpublished-row age directly; coordinate alerting thresholds with whoever operates Debezium.

4.6 Order-Status Consumer scaling. Risk: unlike the Dispatcher/Initiator split (2.4), this consumer has no designed parallelism story — could bottleneck if `order-data` volume grows. Mitigation: a consumer group parallelizes across `order-data`'s existing partitions, but we don't control that topic's partitioning — needs confirmation with the Trading/Orders team.

4.7 Rejection reasons reaching the user unclearly. Risk: since cash isn't pre-checked (2.8), the only signal is `OrderRejected`'s reason code — if it's generic, neither the user nor Notification can give a specific, actionable message. Mitigation: confirm with Trading/Orders that the reason code is structured, not free text (5.1 #2).

4.8 Event-schema coupling with Notification. Risk: enriching `ExecutionProcessed/PlanDeactivated` for Notification (2.11) means our internal events now carry externally-consumed fields — a harmless-looking internal refactor could silently break a consumer we don't own. Mitigation: treat published event shapes as a versioned contract once external consumers exist — additive changes only, explicit deprecation window.